



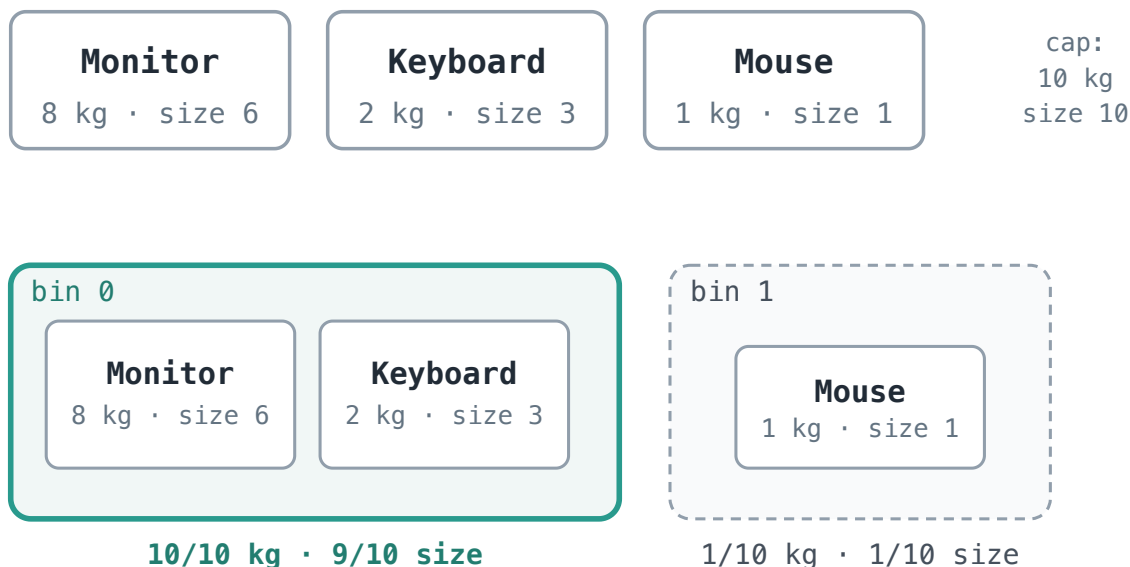
Problem Breakdown

Inventory Packer 🏠

⚡ Greedy & Bin Packing

Published Feb 16, 2025 · 🟢 easy

You're given a list of items, each with a weight and a size. The warehouse hands out bins with two caps, a weight limit and a size limit. Assign every item to a bin such that no bin exceeds either limit. The output is a map from item name to the bin id that holds it. The goal is to minimize the number of bins.



Three items packed into two bins. Bin 0 holds Monitor (8kg, 6size) and Keyboard (2kg, 3size) filling the weight cap, and bin 1 holds Mouse (1kg, 1size) because the weight cap left no room in bin 0

The rules impose a few constraints worth tracing through. Items can't be split across bins, and both caps apply at the same time, so a bin holding the Monitor and Keyboard above is full on weight even though it still has a sliver of size room left for the Mouse. The warehouse hands you one method to list the bins you've already opened in creation order and another method to open a fresh bin that returns its new id.

The problem is a classic bin-packing problem. Finding the absolute minimum number of bins is NP-hard, which means there's no known polynomial algorithm that always produces the optimum. What we can do is pick a fast heuristic, a strategy that usually produces a near-

optimal answer even though it gives up the guaranteed optimum, that produces a near-optimal count on the inputs we care about.

⚡ Pattern: Greedy & Bin Packing

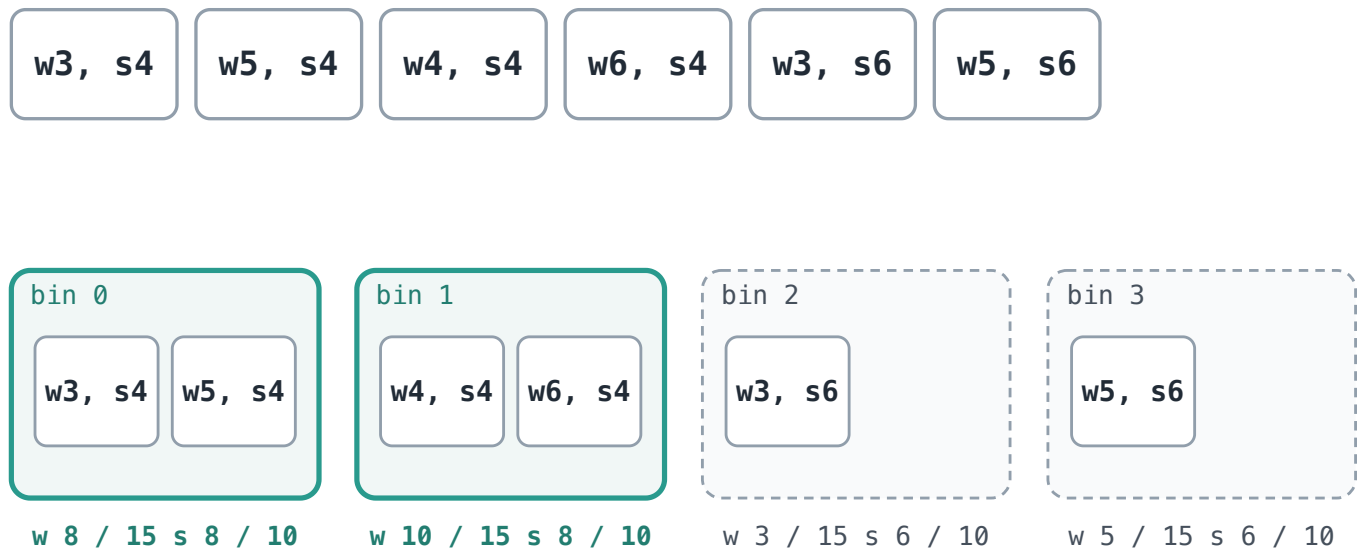
First-Fit and First-Fit Decreasing are classic greedy bin-packing heuristics. You place each item in the first bin it fits and never look back, trading a perfect answer for one that's fast and close.

Learn This Pattern

Solution 1, First-Fit

The simplest workable heuristic is First-Fit. For each item in the order you receive it, walk the existing bins and put the item in the first one that still has room for it. If nothing fits, open a brand-new bin for the item and move on to the next one.

This is a pure **greedy** algorithm. There's no ranking, no scoring, no lookahead. The first legal placement wins, and a new bin only opens when no existing bin has room.



First-Fit packing six items into four bins

The diagram shows the six-item example (w_3, s_4) , (w_5, s_4) , (w_4, s_4) , (w_6, s_4) , (w_3, s_6) , (w_5, s_6) packed in input order. The first four items are all size-4. First-Fit pairs them into bins where both caps hold. Bin 0 takes (w_3, s_4) and (w_5, s_4) for a total weight of 8 and size of 8. Bin 1 takes (w_4, s_4) and (w_6, s_4) for weight 10 and size 8. The next item (w_3, s_6) asks for 6 more size units, but bin 0 and bin 1 already hold size 8, so $8 + 6 = 14$ blows past the size cap of 10. It opens bin 2. The last item (w_5, s_6) runs into the same wall on bins 0 and 1, and also on bin 2 because $6 + 6 = 12$ still exceeds the size cap. It opens bin 3. The result is four bins to hold six items.

 First-FitPython  

```
def pack(self, items: List[Item]) -> Dict[str, int]:
    assignment: Dict[str, int] = {}
    for item in items:
        placed = False
        for b in self.warehouse.get_bins():
            if b.can_fit(item):
                b.add_item(item)
                assignment[item.name] = b.bin_id
                placed = True
                break
        if not placed:
            new_bin = self.warehouse.create_bin()
            new_bin.add_item(item)
            assignment[item.name] = new_bin.bin_id
    return assignment
```

A few details in the code are worth tracing through. The fit check covers both caps at once. The method compares the candidate's weight against the bin's remaining weight and its size against the bin's remaining size, and both comparisons have to pass for the placement to go through. If either cap is the binding constraint the item gets rejected and the loop moves on to the next bin. Once the item lands somewhere, the `break` stops the walk; without it the loop would keep checking later bins even though the item is already placed, and depending on how the assignment is recorded you can end up writing down a later bin id instead of the one the item actually went into. The "open a new bin" branch is where candidates most often drop a step. The warehouse hands you a fresh empty bin and its id, but it doesn't add the item for you. You have to add the item to the new bin yourself and then record the assignment against that bin's id. Forgetting either one is a common bug and the tests catch it.

Complexity

Let n be the number of items and b the number of bins opened so far. Each item triggers at most one walk through the existing bins, which costs $O(b)$ (meaning the work grows linearly with b , so each item is bounded by one pass over the current bin list), plus a potential new-bin call at the end. Placing all n items is $O(n \cdot b)$ overall.

In the worst case every item opens its own bin, so $b = n$ and the total cost is $O(n^2)$. In practice bins fill up, b stays small compared to n , and the loop is close to linear. The runtime is fine either way. Speed is not what this problem is testing.

Where it breaks

First-Fit passes the basic test and the four-identical-item test without issue. The failure shows up on the large-catalog test, which runs thirty items through a warehouse with caps (weight 15, size 10). The catalog is adversarial. The first fifteen items have size 4, and the last fifteen have size 6.

First-Fit sees the size-4 items first. Two size-4 items total size 8, which fits. Three size-4 items total size 12, which doesn't. So First-Fit fills the first seven bins with pairs of size-4 items, each ending at size 8 and comfortably under the weight cap. Bin 7 ends up holding the last size-4 item and the first size-6 item (size $4 + 6 = 10$, exactly at the cap).

Now the remaining fourteen size-6 items arrive. Every existing bin with a size-4 pair already holds size 8, and $8 + 6 = 14$ exceeds the size cap. None of them can accept a size-6 item. Each size-6 item opens its own fresh bin and sits there alone. The final count is 22 bins against a budget of 18.

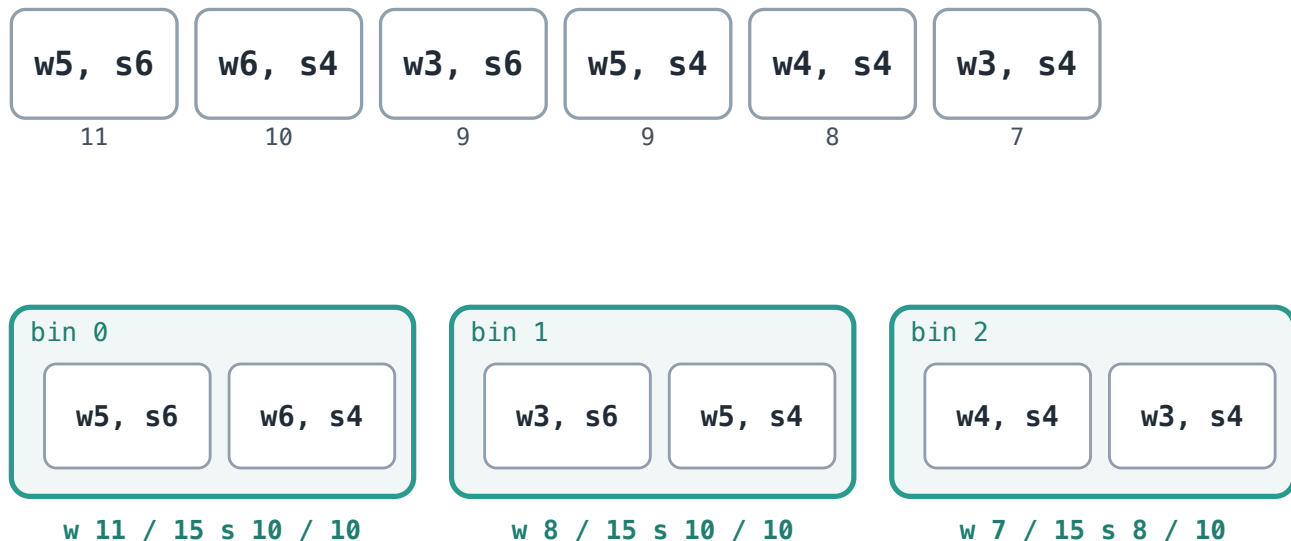
The code is correct and the caps are respected. It just committed to a bunch of "almost full" bins early on that end up having room for nobody.

Solution 2, First-Fit Decreasing

Sort the items first. First-Fit got into trouble because it filled bins to size 8 with small items before the large items arrived. If the large items had gone in first, they would have opened bins at size 6, and the small items would then slot into the remaining 4 units of each bin. The items

are the same and the bin cap is the same, but the result changes completely because the order changed.

That sort order has a name. First-Fit Decreasing (FFD) sorts the items by footprint in descending order and then runs the exact same First-Fit loop. The heuristic for "what's the footprint" is `weight + size`. Size alone would also work on this particular dataset because size is what runs out first, but summing both caps keeps the key reasonable when weight is the binding constraint instead. Imagine a catalog of dumbbells where every item is `(w8, s1)` with a cap of `(w10, s20)`. A size-only key would treat them as tied and leave the order up to input arrival, while `weight + size` still ranks heavier dumbbells ahead of lighter ones and gives the heavy items first crack at empty bins. Neither key is universally optimal, and if you need something more principled you can normalize each axis by its cap (divide weight by the weight cap and size by the size cap so both land in `[0, 1]`, then sum them), but `weight + size` is a simple choice that behaves reasonably on catalogs where either axis can bind (the binding constraint being whichever one runs out of capacity first).



First-Fit Decreasing packing the same six items into three bins

The diagram shows the same six items from before, sorted by `weight + size` descending so the sequence runs `(w5, s6)=11`, `(w6, s4)=10`, `(w3, s6)=9`, `(w5, s4)=9`, `(w4, s4)=8`, and `(w3, s4)=7`. Note that `(w3, s6)` and `(w5, s4)` tie at 9. Which one lands first depends on Python's stable sort keeping the original input order among ties, and other languages' sort stability behaves the same way. Running First-Fit on this order produces three bins instead of

four. Bin 0 gets (w5, s6) and (w6, s4), filling to weight 11 and size 10. Bin 1 gets (w3, s6) and (w5, s4), filling to size 10. Bin 2 gets the two smallest items. Every bin holds a pair because the larger items opened the bins at size 6 and the smaller items topped them off.

First-Fit Decreasing

 Python  

```
def pack(self, items: List[Item]) -> Dict[str, int]:
    sorted_items = sorted(items, key=lambda i: i.weight + i.size, reverse=True)
    assignment: Dict[str, int] = {}
    for item in sorted_items:
        placed = False
        for b in self.warehouse.get_bins():
            if b.can_fit(item):
                b.add_item(item)
                assignment[item.name] = b.bin_id
                placed = True
                break
        if not placed:
            new_bin = self.warehouse.create_bin()
            new_bin.add_item(item)
            assignment[item.name] = new_bin.bin_id
    return assignment
```

A few details in this version are worth walking through. The code copies `items` before sorting rather than sorting in place, because the caller handed us that list and expects it unchanged. Mutating it would be a side effect the tests don't expect. The entire engine below the sort is the same First-Fit code from the previous solution. All that changed is the order the items arrive in, and that one change is what takes the bin count from 22 to 15. The sort key itself, `weight + size`, is a simple choice that handles catalogs where either axis might bind. Other reasonable keys include `max(weight, size)` or weighted combinations that account for cap ratios, but `weight + size` is the shortest thing you can write that still doesn't ignore an entire axis.

Why descending, not ascending

Sorting ascending would actually make things worse, since small-items-first is roughly what the unsorted catalog already looked like. The small items pair up, fill bins to size 8, and leave no room for the large items that arrive later. Descending order flips the problem on its head so that large items open the bins and small items fill the gaps that are left.

Large items are harder to place because they need bins that are mostly empty, while small items can slip into almost any gap a larger item leaves behind. Placing the hard cases first while bins are empty is the strategy that gives them the best chance to land cleanly.

Complexity

The sort is $O(n \log n)$. The First-Fit loop is the same $O(n \cdot b)$ as before, where b is the number of bins opened. The total is $O(n \log n + n \cdot b)$, which collapses to $O(n^2)$ in the worst case when every item opens its own bin and $b = n$. In practice bins fill up, b stays small, and the sort ends up being the dominant term. Either way this is the same order of magnitude as First-Fit with a small additive overhead, and the sort is not the expensive part of the solution.

Why this passes

On the thirty-item catalog FFD produces 15 bins, well under the 18-bin budget. Every single bin holds a pair of items, and every bin is filled to size 10 or size 8. There are no half-empty bins waiting for items that can't arrive. On the eight-item basic-packing catalog FFD produces 3 bins, compared to the 4 First-Fit used. On the four-identical-item test it uses 2 bins and ties First-Fit because there's no ordering advantage when all items are the same.



FFD isn't optimal. For classic one-dimensional bin packing there's a proof that FFD uses at most $(11/9) \cdot \text{OPT} + 6/9$ bins, where the approximation ratio tells you how close a heuristic's answer is to the provable minimum. OPT is that minimum bin count, and this bound says FFD's answer is within about 22% of OPT in the worst case plus a small additive constant. That bound assumes a single cap, though. This problem has two caps (weight and size) and sorts by $\text{weight} + \text{size}$, so the classic proof doesn't directly apply and the gap can in principle be larger. The practical takeaway is the same either way. FFD is provably close to optimal on standard bin packing and behaves well on realistic two-cap

Show More ▾

Benchmarks

Bin counts across the three shipping test datasets.

Dataset	Items	First-Fit bins	FFD bins
basic catalog	8	4	3
four identical (7, 5)	4	2	2
adversarial catalog	30	22	15

First-Fit is fine whenever the input isn't ordered against you. On the basic catalog it uses 4 bins and FFD uses 3, a small win but not a dramatic one. On the four-identical-item fixture both heuristics tie at 2 bins, because when every item has the same footprint there's no sort order that changes anything.

The 22-to-15 gap on the adversarial catalog is where the test lives. The catalog is deterministic and was designed to exploit First-Fit's failure mode. Any time an input has a mix of "blocks bins from pairing" items and "needs empty bins" items, unsorted First-Fit ends up in this shape.

Takeaways

If you got stuck on this problem, the framing that helps most is that each solution picks a different thing to vary. First-Fit varies only the placement rule and treats the input order as fixed. That works when the input isn't pathological but breaks when a realistic warehouse feed sends you the small items first. First-Fit Decreasing keeps the same placement rule and varies the input order instead, and that single change is what takes the bin count from 22 to 15 on the hard cases.

The "sort before you run the greedy" move shows up across the greedy family. Interval scheduling (the classic problem of packing non-overlapping meetings into a room) sorts by end time so the earliest-finishing job locks in first. Whenever a greedy algorithm has a known failure mode on adversarially-ordered inputs, preprocessing with a sort is often the cheapest fix.

One more variant on bin packing is worth naming. **Best-Fit Decreasing** is the same sort-then-loop shape, but instead of placing each item in the *first* bin that accepts it, you place it in the

bin that would leave the *least* remaining room. Best-Fit tends to be a touch better than First-Fit on average, and on standard one-dimensional bin packing it shares a similar worst-case approximation guarantee to FFD (with the same caveat as before that this two-cap variant with a summed sort key isn't directly covered by the classic proof). For this problem FFD passes every test, so there's no reason to reach for it. In an interview, naming Best-Fit and the branch-and-bound escape hatch for exact optimality is the signal you want to send.